TRAINING · HEALTH

FitnessPro

Dark-locked and card-free — built to be read one-handed, mid-set, in about two seconds.

SCOPE
7 screens, built and shot on device

STACK
SwiftUI · Swift 6 · async/await · iOS 26

STANDARD
Light + dark · Dynamic Type · VoiceOver

Swift · SwiftUI · HealthKit · async/await · Swift Charts

> Self-initiated demo build. Designed and coded end to end by me to show current iOS 26 / SwiftUI standards — not client work, and no client material appears in it.

11 yrs	**Fintech**	**SwiftUI**	**7**
iOS engineering	Banking-grade discipline	iOS 26 · Swift 6	Screens in this build

The problem

A training app is read mid-set, one-handed, with about two seconds of attention. Anything that needs a second look has already failed.

What I built

Dark-locked by design, with no cards at all — the rings and the heart-rate zone bands are the surfaces. One giant metric, then a dense set grid, and nothing in between.

02 Engineering decisions

- Progress rings, HR zone bands and a vertical zone bar
- Set/rep grid, weight ruler and rest timer as first-class controls
- Exercise demos: two public-domain frames crossfaded, gated behind prefers-reduced-motion
- Consistency matrix and pace ladder for the history screen
- Accent derives from one hex — the whole zone ramp re-maps with it

Swift · SwiftUI · HealthKit · async/await · Swift Charts · WidgetKit · Mint #3DDC84 on near-black

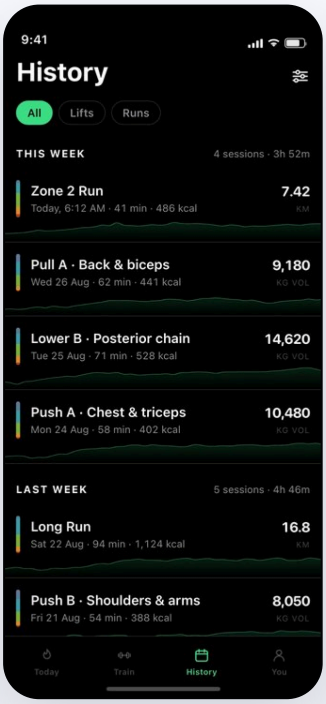
What this transfers to your build

Built for a phone read one-handed, mid-activity, with sweat on the screen: large hit targets, glanceable numbers, and live session state that survives backgrounding. Directly applicable to fitness, field-service, logistics or any app used while the user is doing something else.

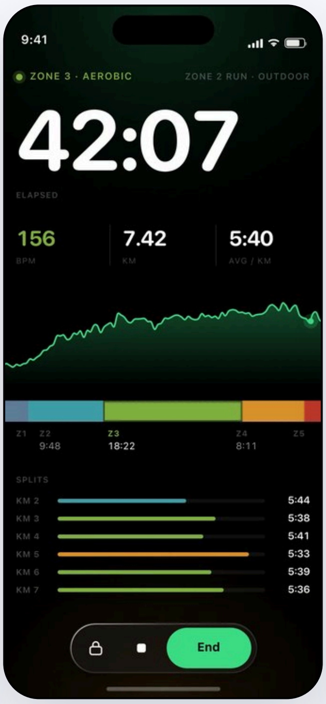
03 Every screen, shipped

7 screens, built in SwiftUI and captured on device. Light and dark, Dynamic Type and VoiceOver labels are wired on each one.

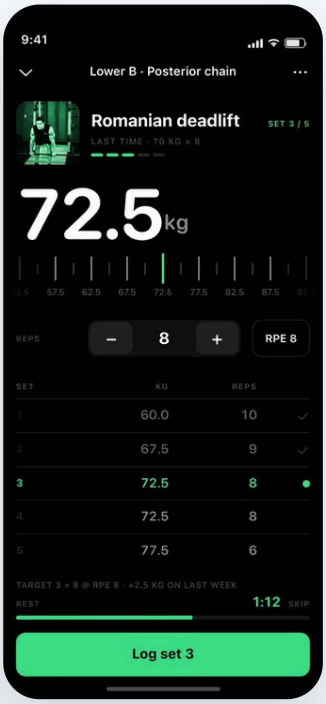
Today's rings



Workout history

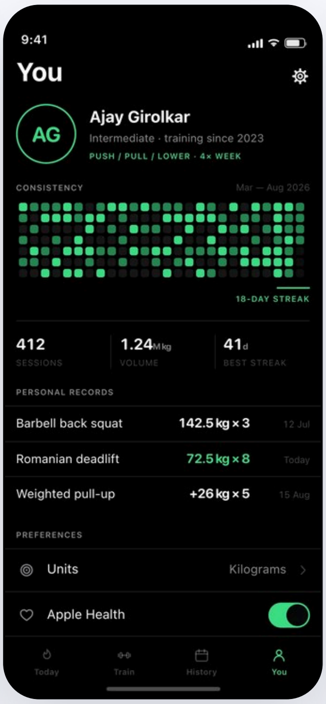


Active workout

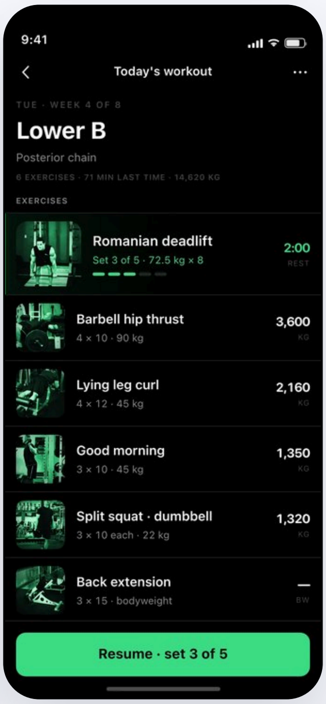


Set logger

Streak celebration



Athlete profile



Workout detail

04 What a build like this gets you

- Production SwiftUI screens, not clickable prototypes — code your team can ship
- MVVM + Clean Architecture with a design token layer: colors, type, spacing, radii
- async/await networking, actors and structured concurrency. No third-party UI deps
- Light and dark, Dynamic Type, VoiceOver labels and reduced-motion paths from day one

05 How we can work together

Screen build
3 screens
6 days

3 SwiftUI screens from your designs, with the token layer and mock data.

Feature or MVP
MOST PICKED

6+ screens
12 days

6+ screens, MVVM, navigation stack, async/await API layer, light and dark.

Full app or rescue
End to end
3 weeks+

End to end build, or an audit and rescue of an inherited codebase, to App Store.

06 Also on the table

- UIKit / Objective-C → SwiftUI migration, Swift 6 strict concurrency
- On-device AI with Apple Foundation Models, Core ML, Vision
- Claude and OpenAI integrations with streaming, cost control and privacy
- StoreKit 2 subscriptions, Widgets, Live Activities, App Intents and Siri
- Crash, performance and App Store rejection fixes on an existing app
- XCTest, Fastlane, Xcode Cloud and GitHub Actions CI/CD

07 How the build runs

DAY 0

Requirements
You send designs or a reference app, target iOS version and API docs. The clock starts here, not the day we agree to work together.

DAY 2

Foundation
Design tokens, navigation shell and data models land. You get a build you can run on your own device.

DAY 6

Screens
Feature screens built against real or mock data, pushed to your repo daily. No silent weeks.

HANDOFF

Ship
Code walkthrough on Loom, one revision round, and a checklist for App Store submission.

Send me the app you are trying to build

Message me with what you are working on — the screens, the deadline, and where it is stuck. I will tell you straight whether I am the right fit, what it takes and how long it runs. Reply the same day, most days.